Vive Zarzal

Aplicación web de foro comunitario para compartir platos y recetas

Integrantes

Cristian Camilo Cortes Ortiz - 202478542
Luisa María Noreña Rueda - 202460789
Katherine Arboleda Ocoró - 202478504

Tecnología en Desarrollo de Software
Universidad del Valle – Regional Zarzal
Desarrollo de Software II

Docente: Gustavo Adolfo Osorio

1 de noviembre de 2026

Tabla de contenido

1. Descripción general
2. Planteamiento del problema
2.1. Pregunta problema
3. Justificación
4. Solución propuesta
4.1. Naturaleza no comercial de la plataforma
4.2. Arquitectura y tecnologías
5. Objetivos
5.1. Objetivo general
5.2. Objetivos específicos
6. Alcance del proyecto
6.1. Incluido en el alcance
6.2. Extensiones preparadas en el modelo de datos
6.3. Fuera del alcance
6.4. Criterios de éxito
7. Actores del sistema
8. Módulos funcionales
9. Requisitos funcionales iniciales
10. Requisitos no funcionales iniciales
11. Riesgos y decisiones pendientes
12. Historias de usuario
13. Reglas de negocio
14. Modelo de datos y entidades
15. Rutas y páginas de la aplicación
16. Configuración, pruebas y plan de trabajo
17. Matriz de trazabilidad

1. Descripción general

Vive Zarzal es una aplicación web que funciona como un foro comunitario gastronómico. Los usuarios registrados publican platos y recetas, descubren las publicaciones de otras personas y participan mediante me gusta, valoraciones de 1 a 5 y comentarios.
La unidad central de contenido es la **publicación**, que puede ser de dos tipos: un **plato** (título, descripción, categoría e imagen opcional) o una **receta**, que además incluye ingredientes e instrucciones de preparación. Cada usuario tiene un perfil básico y administra sus propias publicaciones y comentarios.

El nombre Vive Zarzal se conserva del repositorio original, que fue refactorizado de una plataforma de comercio a un foro. La aplicación no es una tienda virtual: no vende ni compra alimentos, productos o servicios, y no procesa pagos. Su propósito es ofrecer un espacio sencillo para compartir y descubrir contenido gastronómico.

2. Planteamiento del problema

Las personas que disfrutan cocinar y la gastronomía necesitan un espacio sencillo donde compartir sus platos y recetas, descubrir ideas de otras personas e intercambiar opiniones. Sin un sitio dedicado, el contenido queda disperso y es difícil de encontrar, de organizar y de valorar.
Cuando alguien quiere participar en una comunidad de este tipo, puede tener dificultades para:

- Publicar un plato o una receta de forma estructurada, con su categoría, descripción e imagen.
- Encontrar contenido organizado por tipo de comida.
- Distinguir un plato descrito de una receta que incluye ingredientes y pasos de preparación.
- Dar y recibir retroalimentación mediante me gusta, valoraciones y comentarios.
- Mantener y editar sus propias publicaciones sin que otras personas puedan modificarlas.
- Contar con un perfil público desde el cual otras personas conozcan sus publicaciones.

Vive Zarzal responde a esta necesidad con perfiles de usuario, publicaciones categorizadas e interacciones sociales básicas.

2.1. Pregunta problema

¿Cómo ofrecer a una comunidad un espacio web sencillo y mantenible donde las personas puedan publicar platos y recetas, descubrir contenido por categorías e interactuar mediante me gusta, valoraciones y comentarios, garantizando que cada usuario controle únicamente su propio contenido?

3. Justificación

El proyecto propone una aplicación pequeña, comprensible y mantenible que sirva de base para el desarrollo académico continuo. Reúne en un solo sitio el registro de usuarios, la publicación de contenido gastronómico, su exploración por categorías y la interacción entre miembros de la comunidad.

El alcance se limita a un producto funcional para fines académicos. No se pretende operar como un servicio comercial en producción ni establecer compromisos de disponibilidad. Las decisiones técnicas privilegian la simplicidad: una aplicación renderizada en el servidor, una base de datos relacional y una estructura que un equipo universitario pequeño pueda entender y ampliar.

4. Solución propuesta

Desarrollar una aplicación web renderizada en el servidor que integre las siguientes capacidades:

1. **Gestión de usuarios y acceso:** registro, inicio y cierre de sesión, con autenticación mediante token firmado y rutas protegidas.
2. **Perfiles:** perfil público con las publicaciones del usuario y edición del perfil propio.
3. **Publicaciones:** creación, consulta, edición y eliminación de platos y recetas, con imagen opcional.
4. **Categorías y exploración:** catálogo de categorías gastronómicas, feed de publicaciones recientes y feed por categoría.
5. **Interacción comunitaria:** me gusta, valoraciones de 1 a 5 con promedio, y comentarios.
6. **Control de propiedad:** solo el autor modifica o elimina sus publicaciones y comentarios.
7. **Validación y errores:** validación de entradas en el servidor y manejo centralizado de errores.
8. **Base de datos relacional:** modelo normalizado con migraciones, restricciones de unicidad e integridad referencial.

El modelo de datos incluye además tablas para extensiones futuras (roles y permisos, etiquetas, favoritos, notificaciones, reportes, documentos de identidad y ubicaciones). Estas no forman parte del flujo mínimo del foro y solo se desarrollarán si se aprueban expresamente.

4.1. Naturaleza no comercial de la plataforma

Vive Zarzal es un foro de contenido gastronómico. No incluye catálogos de productos para la venta, carritos, pedidos, pagos, facturas, inventarios, envíos ni precios o existencias. Cualquier funcionalidad de comercio queda excluida y no debe reintroducirse en el proyecto.

4.2. Arquitectura y tecnologías

La aplicación sigue una arquitectura de servidor con vistas renderizadas: el navegador solicita páginas a una aplicación Express, que combina routers, controladores, middleware, esquemas de validación y modelos de base de datos. Los controladores asumen directamente los flujos de operaciones simples; no se añadió una capa de servicios porque no aportaba beneficio a los flujos CRUD.

Capa	Tecnología
Ejecución	Node.js 18 o superior
Framework backend	Express 5
Motor de vistas	EJS
ORM	Sequelize 6
Base de datos	PostgreSQL (imagen 17 en Docker Compose)
Administración de la base de datos	pgAdmin
Autenticación	JSON Web Token (JWT) en cookie
Hash de contraseñas	bcryptjs
Validación	Joi
Carga de archivos	Multer
Configuración	dotenv
Desarrollo y calidad	nodemon, ESLint, ejecutor de pruebas integrado de Node.js

5. Objetivos

5.1. Objetivo general

Desarrollar una aplicación web de foro comunitario en la que los usuarios registrados publiquen platos y recetas, exploren contenido por categorías e interactúen mediante me gusta, valoraciones y comentarios, con control de acceso y validaciones en el servidor.

5.2. Objetivos específicos

- Permitir el registro, la autenticación y la gestión básica del perfil de los usuarios.
- Permitir crear y administrar publicaciones propias, distinguiendo platos de recetas.
- Organizar las publicaciones mediante categorías y ofrecer feeds públicos y por categoría.
- Implementar interacciones de me gusta, valoración de 1 a 5 y comentarios, con restricciones que impidan duplicados.
- Ofrecer perfiles públicos con la lista de publicaciones de cada usuario.
- Diseñar un modelo de datos relacional con migraciones, claves foráneas y restricciones de integridad.
- Garantizar controles de autenticación y de propiedad de los recursos, validación de entradas y manejo centralizado de errores.
- Verificar los flujos esenciales mediante pruebas repetibles y documentar la instalación y el uso del proyecto.

6. Alcance del proyecto

6.1. Incluido en el alcance

- Registro, inicio y cierre de sesión.
- Perfil básico de usuario, perfil público y lista de publicaciones del usuario.
- Feed de publicaciones recientes.
- Publicaciones de tipo plato y receta.
- Carga de imagen para las publicaciones.
- Categorías gastronómicas y feed por categoría.
- Crear, consultar, editar y eliminar publicaciones propias.
- Me gusta.
- Valoraciones de 1 a 5.
- Comentarios.
- Verificación de autenticación y de propiedad de los recursos.
- Validación de entradas y manejo centralizado de errores.
- Pruebas, documentación y configuración reproducible del entorno de desarrollo.

6.2. Extensiones preparadas en el modelo de datos

El esquema contiene migraciones y modelos para las siguientes áreas. No forman parte del flujo mínimo del foro y su flujo de aplicación está pendiente:

- Roles y permisos de acceso.
- Documentos de identidad y su verificación.
- Países, departamentos, ciudades y direcciones de usuario.
- Etiquetas asociadas a las publicaciones.
- Publicaciones guardadas o favoritas.
- Notificaciones dentro de la aplicación.
- Reportes y moderación de contenido.

6.3. Fuera del alcance

El proyecto es un foro comunitario, no un marketplace. Quedan excluidos:

- Tiendas de emprendedores o vendedores y registro de negocios.
- Catálogos de productos para la venta, precios y existencias.
- Inventarios, carritos, pedidos, compras y ventas.
- Pasarelas y procesamiento de pagos, facturas y transacciones comerciales.
- Entregas, envíos y flujos de checkout.
- Paneles comerciales de marketplace.

También quedan fuera del alcance actual, salvo aprobación expresa:

- Chat en tiempo real o mensajería privada.
- Redes de seguidores.
- Transmisiones en vivo y alojamiento de video.
- Clases de cocina en línea.
- Planificación de comidas.
- Asesoría nutricional o médica.

- Reservas en restaurantes y pedidos de comida en línea.
- Envío de notificaciones por correo o push.
- Recuperación de contraseña y verificación de correo.
- Búsqueda y paginación.

6.4. Criterios de éxito

El producto base se considerará funcionalmente completo cuando se cumpla lo siguiente:

- Un visitante puede registrarse e iniciar sesión.
- Un usuario autenticado puede crear publicaciones de tipo plato y receta.
- Los usuarios pueden recorrer el feed y los feeds por categoría, y ver una publicación con los detalles de la receta.
- Los usuarios autenticados pueden dar me gusta, valorar y comentar.
- Un usuario solo puede editar o eliminar sus propias publicaciones y comentarios.
- Los usuarios pueden ver perfiles públicos y editar el propio.
- La base de datos impide me gusta y valoraciones duplicadas del mismo usuario sobre la misma publicación.
- La aplicación se instala y ejecuta con el flujo de desarrollo documentado.
- Los flujos esenciales cuentan con pruebas repetibles.

7. Actores del sistema

7.1. Visitante no autenticado

Es una persona que accede a la aplicación sin haber iniciado sesión. Puede consultar las páginas públicas y explorar el contenido público. Debe autenticarse para realizar acciones que modifiquen datos o representen su identidad.

- Registrarse e iniciar sesión.
- Ver el feed, los feeds por categoría y el detalle de las publicaciones.
- Leer los comentarios.
- Consultar perfiles públicos.

7.2. Miembro registrado

Es la persona que usa las funciones de publicación e interacción.

- Iniciar y cerrar sesión.
- Consultar y actualizar su perfil.
- Crear, editar y eliminar sus propias publicaciones de tipo plato o receta, con imagen opcional.
- Dar y quitar me gusta, valorar de 1 a 5 y comentar publicaciones.
- Editar y eliminar sus propios comentarios.

7.3. Administrador (fase futura)

Es un rol previsto en el catálogo de roles. Su flujo de trabajo y el middleware de autorización asociado no forman parte del alcance mínimo. Su responsabilidad prevista es administrar la

plataforma: roles, permisos, categorías, etiquetas y reportes.

7.4. Moderador (fase futura)

Es un rol previsto en el catálogo de roles. Su responsabilidad prevista es revisar y resolver los reportes de contenido. El flujo de moderación está pendiente.
La documentación de datos describe tres roles sembrados: *admin*, *moderator* y *member*. Debe verificarse que esos registros existan en la base de datos de ejecución.

7.5. Otros interesados

Interesado	Interés o responsabilidad
Propietario del producto	Define el propósito, las prioridades y la aceptación del alcance.
Equipo de desarrollo	Implementa, prueba, documenta y mantiene la aplicación.
Docente evaluador	Evalúa el proyecto frente a los objetivos académicos y los entregables acordados.

8. Módulos funcionales

8.1. Autenticación y perfiles

Gestiona el registro, el inicio y cierre de sesión, la sesión mediante JWT en cookie, el perfil propio, el perfil público y la protección de rutas.

8.2. Publicaciones

Gestiona las publicaciones de tipo plato y receta: creación, detalle, edición, eliminación, carga de imagen y validación según el tipo.

8.3. Categorías y descubrimiento

Mantiene el catálogo de categorías y ofrece el feed principal y el feed por categoría. La búsqueda por texto, la paginación y las etiquetas quedan como mejoras futuras.

8.4. Interacción comunitaria

Gestiona los me gusta, las valoraciones con su promedio y los comentarios, junto con las restricciones de propiedad y de unicidad.

8.5. Administración y extensiones

Agrupa las capacidades preparadas en el modelo de datos pero fuera del flujo mínimo: roles y permisos, administración, favoritos, notificaciones, reportes y moderación, documentos de

identidad y ubicaciones.

8.6. Infraestructura y calidad

Comprende el entorno de desarrollo con Docker Compose, las migraciones y semillas de base de datos, las variables de entorno, las pruebas automatizadas, el análisis estático y la documentación.

9. Requisitos funcionales iniciales

Estos requisitos son preliminares y se refinarán mediante casos de uso y criterios de aceptación. La prioridad Baja identifica capacidades de fase futura.

Código	Requisito	Prioridad
RF-01	El sistema permitirá a un visitante registrarse con nombre, apellido, nombre de usuario, correo, contraseña y confirmación de contraseña.	Alta
RF-02	El sistema rechazará el registro cuando el nombre de usuario o el correo ya estén en uso.	Alta
RF-03	El sistema validará los datos de registro antes de crear la cuenta.	Alta
RF-04	El sistema almacenará las contraseñas mediante hash.	Alta
RF-05	El sistema permitirá iniciar sesión con nombre de usuario o correo y contraseña.	Alta
RF-06	El sistema rechazará credenciales inválidas sin revelar si el usuario o el correo existen.	Alta
RF-07	El sistema mantendrá la autenticación con un JWT firmado almacenado en una cookie HTTP-only.	Alta
RF-08	El sistema permitirá cerrar sesión.	Alta
RF-09	El sistema cargará el usuario actual en las solicitudes que incluyan una cookie de autenticación válida.	Alta
RF-10	El sistema protegerá las rutas restringidas frente a usuarios no autenticados.	Alta
RF-11	El sistema asignará el rol estándar *member* a las cuentas nuevas.	Media
RF-12	El sistema permitirá la recuperación de contraseña y la verificación de correo.	Baja
RF-13	El sistema permitirá a los visitantes ver el perfil público de un usuario.	Alta
RF-14	El perfil público mostrará la información básica del usuario y sus publicaciones.	Alta

Código	Requisito	Prioridad
RF-15	Un usuario autenticado podrá abrir el formulario de edición de su perfil.	Alta
RF-16	Un usuario autenticado podrá actualizar su nombre, apellido, correo y biografía.	Alta
RF-17	El sistema impedirá modificar el perfil de otro usuario mediante el flujo de edición de perfil.	Alta
RF-18	El sistema exigirá correo único al actualizar el perfil.	Alta
RF-19	El sistema mostrará un feed público de publicaciones recientes.	Alta
RF-20	El sistema admitirá publicaciones de tipo plato (*dish*) y receta (*recipe*).	Alta
RF-21	Un usuario autenticado podrá crear una publicación de tipo plato.	Alta
RF-22	Un usuario autenticado podrá crear una publicación de tipo receta.	Alta
RF-23	Una publicación contendrá título, descripción, categoría, tipo, autor e imagen opcional.	Alta
RF-24	Una receta contendrá ingredientes e instrucciones de preparación.	Alta
RF-25	El sistema permitirá a un visitante ver el detalle de una publicación.	Alta
RF-26	El autor podrá editar su propia publicación.	Alta
RF-27	El autor podrá eliminar su propia publicación.	Alta
RF-28	El sistema impedirá que un usuario edite o elimine publicaciones de otro usuario.	Alta
RF-29	El sistema validará los datos de la publicación según su tipo.	Alta
RF-30	El sistema aceptará los formatos de imagen admitidos y aplicará el límite de tamaño configurado.	Media
RF-31	El sistema mantendrá un catálogo de categorías gastronómicas.	Alta
RF-32	Cada publicación pertenecerá a una sola categoría.	Alta
RF-33	Los visitantes podrán explorar las publicaciones por categoría.	Alta
RF-34	Las direcciones de categoría usarán un identificador legible (*slug*).	Media
RF-35	El sistema admitirá etiquetas y filtrado por etiquetas.	Baja
RF-36	El sistema admitirá búsqueda de publicaciones y paginación.	Baja
RF-37	Un usuario autenticado podrá dar me gusta a una publicación.	Alta
RF-38	Un usuario autenticado podrá quitar su me gusta.	Alta
RF-39	El sistema impedirá me gusta duplicados del mismo usuario sobre la misma publicación.	Alta

Código	Requisito	Prioridad
RF-40	El sistema mostrará el número de me gusta de una publicación.	Media
RF-41	Un usuario autenticado podrá valorar una publicación con un entero de 1 a 5.	Alta
RF-42	Un usuario tendrá como máximo una valoración activa por publicación.	Alta
RF-43	Un usuario podrá actualizar su valoración existente.	Alta
RF-44	El sistema calculará y mostrará la valoración promedio de una publicación.	Alta
RF-45	Un usuario autenticado podrá comentar una publicación.	Alta
RF-46	Los visitantes podrán leer los comentarios de una publicación.	Alta
RF-47	El autor de un comentario podrá editarlo.	Alta
RF-48	El autor de un comentario podrá eliminarlo.	Alta
RF-49	El sistema impedirá editar o eliminar comentarios de otros usuarios.	Alta
RF-50	El sistema validará el contenido del comentario y aplicará la longitud máxima.	Alta
RF-51	El sistema admitirá roles de usuario.	Baja
RF-52	El sistema admitirá permisos asignados a roles.	Baja
RF-53	El sistema aplicará los permisos mediante middleware de autorización.	Baja
RF-54	Un administrador podrá gestionar roles, categorías, etiquetas y reportes.	Baja
RF-55	Un moderador podrá revisar y resolver reportes de contenido.	Baja
RF-56	Los usuarios podrán guardar y quitar publicaciones favoritas, y ver las guardadas.	Baja
RF-57	El sistema creará notificaciones dentro de la aplicación para interacciones relevantes, y los usuarios podrán verlas y marcarlas como leídas.	Baja
RF-58	Los usuarios podrán reportar una publicación o un comentario.	Baja
RF-59	El sistema podrá mantener tipos de documento y documentos de identidad de los usuarios, y su verificación.	Baja
RF-60	El sistema podrá mantener países, departamentos y ciudades, y los usuarios podrán gestionar direcciones asociadas a su perfil.	Baja

10. Requisitos no funcionales iniciales

- **Seguridad de contraseñas:** las contraseñas nunca se almacenan en texto plano; se usa bcryptjs con 11 rondas de sal.
- **Seguridad de la sesión:** los tokens JWT se firman y se validan antes de usarse. La cookie de autenticación (*authentication*) es HTTP-only, usa SameSite en modo *lax*, es segura en producción y dura 24 horas.
- **Autorización:** las acciones restringidas exigen usuario autenticado, y el backend valida que cada usuario solo modifique los recursos que posee, salvo que un rol administrativo autorizado lo permita.
- **Protección CSRF:** los formularios que cambian datos con autenticación por cookie requieren protección contra CSRF. La documentación de traspaso indica que aún no está implementada.
- **Gestión de secretos:** los secretos se entregan por variables de entorno y no se versionan.
- **Carga de archivos:** las imágenes se restringen por tipo (JPEG, PNG, WEBP y GIF) y por tamaño (5 MB por defecto, configurable). Se recomienda validar además la firma del contenido.
- **Validación:** los datos de entrada se validan en el servidor con esquemas Joi, además de la validación de la interfaz.
- **Integridad:** el esquema define claves primarias, claves foráneas, índices únicos y acciones referenciales. Las migraciones deben poder repetirse en una base de datos limpia.
- **Consistencia de errores:** los fallos devuelven respuestas controladas (por ejemplo 404 y 500) mediante manejo centralizado, sin errores no capturados.
- **Mantenibilidad:** se conserva el stack Node.js, Express, EJS, Sequelize y PostgreSQL; los cambios de esquema se hacen con migraciones; las asociaciones se centralizan en *src/db/models/index.js*; la documentación se actualiza con cada cambio.
- **Usabilidad y accesibilidad:** formularios con mensajes de validación claros, navegación comprensible, texto alternativo en imágenes, elementos accesibles por teclado y diseño adaptable a escritorio, tableta y móvil. No se ha definido un objetivo formal de accesibilidad.
- **Rendimiento:** no se establecen objetivos cuantitativos. Se recomienda evitar consultas innecesarias, usar los índices existentes, calcular el promedio de valoraciones de forma eficiente y añadir paginación antes de que crezca el volumen de datos.
- **Compatibilidad:** Node.js 18 o superior, PostgreSQL 17 mediante Docker Compose (que levanta la base de datos y pgAdmin, no la aplicación) y puerto 3377 por defecto.
- **Pruebas:** verificar autenticación, perfiles, publicaciones, categorías, interacciones, permisos y restricciones de base de datos.
- **Reproducibilidad y documentación:** permitir ejecutar el proyecto desde cero con un entorno de desarrollo documentado, y describir instalación, configuración, rutas y decisiones de diseño.
- **Trazabilidad:** relacionar requisitos, historias de usuario, diseño, implementación y pruebas.

No se establecen objetivos cuantitativos de disponibilidad propios de un sistema en producción, ya que no forman parte del alcance del curso.

11. Riesgos y decisiones pendientes

11.1. Riesgos

Tema	Riesgo
Estado del repositorio	La documentación declara flujos implementados que la instantánea revisada no demuestra (véase 11.3). Antes de planificar, el equipo debe comparar este documento con la rama real.
Datos semilla	Los roles y las siete categorías dependen de seeders que no aparecen en la instantánea. Sin ellos, el registro puede crear usuarios sin rol *member* y no habrá categorías para publicar.
Seguridad	No hay protección CSRF y la autenticación usa cookies. Se recomienda implementarla junto con los formularios que modifican datos, y no como mejora de la última etapa.
Dependencias	Se reportó una vulnerabilidad en producción por la dependencia *uuid* dentro de Sequelize 6, cuya solución requiere una actualización mayor del ORM. Debe revisarse con una auditoría actual.
Crecimiento del alcance	Las tablas de extensiones existen, pero implementarlas sin aprobación desvía al equipo del foro mínimo. El tiempo académico exige acotar funcionalidades.
Almacenamiento de imágenes	Las imágenes se guardan en disco local (*src/public/uploads/*), sin almacenamiento externo.
Pruebas	No existe prueba de extremo a extremo en navegador, y los archivos de pruebas citados en la documentación no aparecen en la instantánea.
Reglas de interacción	Hay reglas sin definir, como si un autor puede valorar o comentar su propia publicación.

11.2. Decisiones pendientes

ID	Decisión por confirmar	Propuesta por defecto
DP-01	¿El nombre definitivo del proyecto es Vive Zarzal?	Conservar Vive Zarzal.
DP-02	¿Se requieren los flujos de administrador y moderador en la entrega del curso?	Mantenerlos como alcance futuro salvo asignación

ID	Decisión por confirmar	Propuesta por defecto
		expresa.
DP-03	¿Son necesarios los documentos de identidad en un foro de comida?	Excluir del alcance mínimo.
DP-04	¿Son necesarias las direcciones y ubicaciones geográficas?	Excluir del alcance mínimo.
DP-05	¿Se requieren etiquetas además de categorías?	Aplazar hasta que las categorías funcionen.
DP-06	¿Se requieren favoritos?	Aplazar; son distintos de los me gusta.
DP-07	¿Se requieren notificaciones?	Aplazar, sin entrega en tiempo real.
DP-08	¿Se requieren reportes y moderación de contenido?	Aplazar salvo que la moderación sea un objetivo académico explícito.
DP-09	¿Puede un autor dar me gusta, valorar o comentar su propia publicación?	Decidirlo y codificarlo como regla de negocio.
DP-10	¿Cuántos decimales se muestran en el promedio de valoraciones?	Dos decimales.
DP-11	¿Los ingredientes e instrucciones son texto simple o listas estructuradas?	Texto simple en el alcance actual.
DP-12	¿Pueden los usuarios cambiar su nombre de usuario?	No incluirlo en la edición de perfil actual.
DP-13	¿La eliminación de publicaciones es lógica o permanente?	Seguir el comportamiento en cascada de la base de datos, salvo que el docente exija recuperación.
DP-14	¿Qué navegadores y dispositivos se deben soportar como mínimo?	Navegadores modernos de escritorio y móvil.
DP-15	¿Existen objetivos formales de rendimiento, disponibilidad o accesibilidad?	Definirlos solo si la rúbrica del curso lo exige.
DP-16	¿Las categorías las administra solo un administrador o son datos semilla fijos?	Catálogo predefinido en el alcance mínimo.
DP-17	¿Los comentarios admiten respuestas anidadas?	No; comentarios planos.
DP-1	¿Qué consultas están disponibles para visitantes no	Lectura pública del feed,

ID	Decisión por confirmar	Propuesta por defecto
8	autenticados?	categorías, detalles, comentarios y perfiles.

11.3. Estado del repositorio frente a la documentación

La línea base de este documento se elaboró a partir de una instantánea del repositorio (Repomix), cuyo encabezado indica que algunos archivos pueden haberse excluido. Por eso, la ausencia de un archivo en la instantánea no prueba que no exista en la rama real (*refactor/food-community-forum*). Las diferencias detectadas son:

Área	Lo que dice la documentación	Lo que muestra la instantánea
Publicaciones	El feed y el CRUD están implementados.	No hay controlador ni router de publicaciones.
Interacciones	Me gusta, valoraciones y comentarios están implementados.	Existen modelos y migraciones, pero no controlador ni router.
Inicio, categorías y perfil público	Existen *GET /*, *GET /categories/:slug* y *GET /users/:id*.	Ninguna de esas rutas está registrada.
Edición de perfil	Está implementada.	Hay router y controlador, pero falta la vista *editProfile*.
Vistas	Existen vistas de publicaciones y formularios.	Solo aparecen *login.ejs* y *register.ejs*.
Pruebas	Hay pruebas de integración y de esquema.	No hay directorio *tests/*, aunque *npm test* apunta a él.
Semillas	Cinco seeders pueblan los catálogos.	No hay directorio *src/db/seeders/*.
Cierre de sesión	Redirige a */*.	No hay ruta raíz registrada.
Rol del usuario nuevo	Las cuentas nuevas reciben el rol *member*.	El controlador lo asigna solo si encuentra el rol.

En conclusión, el repositorio tiene una base de datos sustancial y un código de autenticación funcional, pero la instantánea no permite certificar el flujo completo del foro. El primer paso del equipo es reconciliar esta diferencia.

12. Historias de usuario

Las historias de usuario se organizan según los módulos funcionales de la sección 8. La prioridad indica la importancia de cada historia para la primera versión, y la columna Requisito

la relaciona con los requisitos funcionales de la sección 9. Las historias HU-13 y HU-18 a HU-23 corresponden a capacidades de fase futura.

12.1. Autenticación y usuarios

ID	Historia de usuario	Criterios de aceptación	Prioridad	Requisito
HU-01	**Registro de usuarios** Como visitante, quiero crear una cuenta en Vive Zarzal, para participar en la comunidad gastronómica.	● El sistema permite registrar un usuario con nombre, apellido, nombre de usuario, correo, contraseña y confirmación. ● Valida los campos obligatorios y sus reglas: nombre y apellido de 2 a 80 caracteres, usuario alfanumérico de 3 a 50, correo válido de máximo 150 y contraseña de 8 a 72 caracteres. ● Rechaza el registro si la confirmación no coincide con la contraseña. ● Impide registrar un nombre de usuario o un correo ya existentes. ● La contraseña se guarda con hash. ● Al terminar, confirma el registro, dirige al inicio de sesión y asigna el rol *member*.	Alta	RF-01 a RF-04, RF-11
HU-02	**Inicio de sesión** Como usuario registrado, quiero iniciar sesión con mi nombre de usuario o mi correo, para acceder a las funciones de miembro.	● El sistema acepta nombre de usuario o correo junto con la contraseña. ● Si las credenciales son incorrectas, muestra un error genérico que no revela si la cuenta existe. ● Si son correctas, guarda un JWT en la cookie de autenticación. ● Redirige al destino previsto cuando es seguro; si no lo es, al inicio. ● Las rutas protegidas redirigen a quien no ha iniciado sesión a la página de ingreso y conservan el destino.	Alta	RF-05 a RF-07, RF-09, RF-10

ID	Historia de usuario	Criterios de aceptación	Prioridad	Requisito
HU-03	**Cierre de sesión** Como miembro autenticado, quiero cerrar sesión, para que mi navegador deje de estar autenticado.	● La acción de cierre elimina la cookie de autenticación. ● El sistema redirige a una página pública. ● Las acciones restringidas vuelven a exigir inicio de sesión.	Alta	RF-08
HU-04	**Edición del perfil** Como miembro autenticado, quiero consultar y actualizar mis datos básicos, para mantener mi información al día.	● El sistema carga los valores actuales del usuario autenticado. ● Permite editar nombre, apellido, correo y biografía (hasta 500 caracteres). ● Valida los datos antes de guardar. ● Rechaza un correo que pertenezca a otra cuenta. ● No permite consultar ni modificar el perfil de otro usuario mediante este flujo.	Alta	RF-15 a RF-18
HU-05	**Perfil público** Como visitante, quiero ver el perfil público de un miembro, para descubrir sus publicaciones.	● El perfil muestra la información pública del usuario. ● Lista las publicaciones de ese usuario. ● Un usuario inexistente produce una respuesta de no encontrado.	Alta	RF-13, RF-14
HU-06	**Control de acceso y propiedad** Como responsable del sistema, quiero que las acciones estén protegidas según la autenticación y la propiedad del recurso, para evitar modificaciones no autorizadas.	● Las operaciones que crean o modifican datos exigen un usuario autenticado. ● El backend verifica que el usuario sea el autor antes de editar o eliminar publicaciones y comentarios, incluso si la solicitud se hace directamente. ● Los intentos no autorizados se rechazan con una respuesta controlada. ● Los formularios que modifican datos cuentan con protección CSRF.	Alta	RF-10, RF-17, RF-28, RF-49

12.2. Publicaciones

ID	Historia de usuario	Criterios de aceptación	Prioridad	Requisito
HU-07	**Feed de publicaciones** Como visitante, quiero ver las publicaciones recientes, para descubrir platos y recetas.	● El feed muestra las publicaciones recientes. ● Cada tarjeta muestra título, imagen o marcador de posición, autor, categoría, tipo y un resumen de interacción. ● Si no hay publicaciones, muestra un mensaje de estado vacío. ● El usuario puede abrir el detalle de cada publicación.	Alta	RF-19, RF-25, RF-40, RF-44
HU-08	**Publicar un plato** Como miembro autenticado, quiero publicar un plato, para compartir lo que cocino.	● El formulario pide título (3 a 150 caracteres), descripción (10 a 2000) y categoría. ● La imagen es opcional: se aceptan JPEG, PNG, WEBP y GIF hasta el tamaño configurado (5 MB por defecto). ● La publicación queda asociada a mi cuenta con tipo *dish*. ● Datos inválidos, categoría inexistente o imagen no permitida impiden la creación. ● Si falla el guardado, se eliminan los archivos temporales subidos.	Alta	RF-20, RF-21, RF-23, RF-29, RF-30
HU-09	**Publicar una receta** Como miembro autenticado, quiero publicar una receta, para que otras personas puedan prepararla.	● Además de los datos de un plato, el formulario exige ingredientes e instrucciones (3 a 3000 caracteres cada uno). ● La publicación se guarda con tipo *recipe*. ● La falta de ingredientes o instrucciones impide la creación. ● El detalle de la receta muestra ingredientes e instrucciones.	Alta	RF-22, RF-24, RF-29, RF-30
HU-10	**Detalle de una publicación**	● El detalle muestra título, descripción, autor, categoría,	Alta	RF-25, RF-46

ID	Historia de usuario	Criterios de aceptación	Prioridad	Requisito
	Como visitante, quiero abrir una publicación, para leer su contenido y ver la opinión de la comunidad.	imagen, me gusta, valoración promedio y comentarios. ● Los ingredientes y las instrucciones se muestran solo en las recetas. ● Una publicación inexistente produce una respuesta de no encontrado. ● Las acciones de editar y eliminar se muestran solo al autor.		
HU-11	**Gestión de mis publicaciones** Como autor, quiero editar o eliminar mis publicaciones, para mantener mi contenido.	● El autor puede editar y eliminar sus publicaciones. ● Otro miembro no puede editarlas ni eliminarlas. ● Al eliminar una publicación se eliminan sus me gusta, valoraciones y comentarios. ● Si el usuario no está autenticado, se le redirige al inicio de sesión. ● Si la validación falla, el registro existente no se pierde.	Alta	RF-26 a RF-28

12.3. Categorías y descubrimiento

ID	Historia de usuario	Criterios de aceptación	Prioridad	Requisito
HU-12	**Explorar por categoría** Como visitante, quiero filtrar las publicaciones por categoría, para encontrar contenido relevante.	● Al elegir una categoría, se muestran solo las publicaciones de esa categoría. ● Las categorías son un catálogo predefinido (por ejemplo desayuno, almuerzo, cena, postres, bebidas, comida tradicional y vegetariana). ● Un identificador de categoría desconocido produce una respuesta de no encontrado. ● Una categoría sin publicaciones muestra un estado vacío.	Alta	RF-31 a RF-34

ID	Historia de usuario	Criterios de aceptación	Prioridad	Requisito
HU-13	**Búsqueda y paginación** Como visitante, quiero buscar publicaciones y recorrerlas por páginas, para ubicar contenido cuando el feed crezca.	● El sistema permite buscar por texto en los campos definidos. ● Los resultados se dividen en páginas. ● La paginación funciona junto con la búsqueda y los filtros. ● Las validaciones se hacen en el backend.	Baja	RF-35, RF-36

12.4. Interacción comunitaria

ID	Historia de usuario	Criterios de aceptación	Prioridad	Requisito
HU-14	**Me gusta** Como miembro autenticado, quiero dar me gusta a una publicación, para expresar que me gustó.	● El miembro puede agregar y quitar su me gusta. ● La base de datos impide me gusta duplicados del mismo usuario sobre la misma publicación. ● El contador refleja los registros existentes. ● Si el usuario no ha iniciado sesión, se le invita a hacerlo.	Alta	RF-37 a RF-40
HU-15	**Valoración** Como miembro autenticado, quiero valorar una publicación de 1 a 5, para dar una opinión estructurada.	● Solo se aceptan enteros de 1 a 5. ● Cada miembro tiene una sola valoración por publicación y puede actualizarla. ● El promedio se calcula con las valoraciones activas y se muestra en la publicación. ● Una valoración inválida, una publicación inexistente o un usuario sin sesión producen un error controlado.	Alta	RF-41 a RF-44
HU-16	**Comentarios** Como miembro autenticado, quiero comentar una publicación,	● El comentario no puede estar vacío y tiene un máximo de 1000 caracteres. ● Cada comentario muestra su autor y su contenido.	Alta	RF-45, RF-46, RF-50

ID	Historia de usuario	Criterios de aceptación	Prioridad	Requisito
	para conversar con su autor y con otros miembros.	● Los visitantes pueden leer los comentarios. ● Los comentarios son planos, sin respuestas anidadas.		
HU-17	**Gestión de mis comentarios** Como autor de un comentario, quiero editarlo o eliminarlo, para corregir o retirar lo que escribí.	● El autor puede editar y eliminar su comentario. ● Otros usuarios no pueden modificarlo. ● Un comentario inexistente produce una respuesta de no encontrado. ● El contenido editado se valida con las mismas reglas que un comentario nuevo.	Alta	RF-47 a RF-49

12.5. Extensiones futuras

Las siguientes historias corresponden a capacidades preparadas en el modelo de datos y no deben tratarse como funcionalidades completadas.

ID	Historia de usuario	Criterios de aceptación	Prioridad	Requisito
HU-18	**Favoritos** Como miembro, quiero guardar publicaciones, para encontrarlas después.	● El miembro puede guardar y quitar una publicación. ● Puede ver la lista de sus publicaciones guardadas. ● Los favoritos son independientes de los me gusta.	Baja	RF-56
HU-19	**Notificaciones** Como miembro, quiero recibir avisos de interacciones con mi contenido, para enterarme de la actividad.	● El sistema crea notificaciones dentro de la aplicación para me gusta, comentarios y valoraciones. ● El miembro puede verlas y marcarlas como leídas. ● No se incluye entrega en tiempo real ni por correo.	Baja	RF-57
HU-20	**Reporte de contenido** Como miembro, quiero reportar	● El miembro puede reportar una publicación o un comentario. ● El reporte conserva quién lo	Baja	RF-58

ID	Historia de usuario	Criterios de aceptación	Prioridad	Requisito
	contenido inapropiado, para que sea revisado.	hizo y qué contenido señala.		
HU-21	**Moderación** Como moderador, quiero revisar y resolver reportes, para mantener un contenido adecuado.	● El moderador ve los reportes pendientes. ● Puede resolverlos y queda registrado quién los revisó. ● La acción exige el permiso correspondiente.	Baja	RF-55
HU-22	**Administración** Como administrador, quiero gestionar roles, permisos, categorías y etiquetas, para configurar la plataforma.	● El sistema aplica permisos por rol mediante middleware de autorización. ● El administrador gestiona roles, permisos, categorías, etiquetas y reportes. ● Los usuarios sin el permiso no pueden ejecutar estas operaciones, ni siquiera llamando a la API directamente.	Baja	RF-35, RF-51 a RF-54
HU-23	**Perfil ampliado** Como miembro, quiero registrar mi ubicación y documentos de identidad, para completar mi perfil.	● El miembro puede gestionar direcciones con selectores de país, departamento y ciudad. ● Puede registrar documentos de identidad, que un administrador verifica. ● Su inclusión depende de que se confirmen DP-03 y DP-04.	Baja	RF-59, RF-60

13. Reglas de negocio

ID	Regla
RN-01	Solo los usuarios registrados pueden realizar acciones autenticadas.
RN-02	A los usuarios registrados se les asigna el rol estándar *member* cuando el registro del rol está disponible.
RN-03	Una publicación tiene un único autor y pertenece a una única categoría.
RN-04	El tipo de una publicación es *dish* o *recipe*.
RN-05	Una receta debe incluir ingredientes e instrucciones de preparación; un plato no los

ID	Regla
	requiere.
RN-06	La imagen de una publicación es opcional.
RN-07	Solo el autor puede editar o eliminar su publicación, salvo que un permiso administrativo o de moderación futuro lo autorice expresamente.
RN-08	Un usuario puede dar como máximo un me gusta por publicación y puede retirar el suyo.
RN-09	Un usuario puede tener como máximo una valoración por publicación; debe ser un entero de 1 a 5 y puede actualizarla.
RN-10	Un comentario pertenece a un usuario y a una publicación.
RN-11	Solo el autor de un comentario puede editarlo o eliminarlo, salvo permiso administrativo o de moderación futuro.
RN-12	El nombre de usuario y el correo son únicos.
RN-13	Una actualización de perfil no puede asignar un correo que ya usa otra cuenta.
RN-14	Una categoría referenciada por publicaciones no puede eliminarse mientras rija la relación *RESTRICT* configurada.
RN-15	Al eliminar una publicación se eliminan, según las claves foráneas, sus me gusta, valoraciones, comentarios y demás registros dependientes.
RN-16	La plataforma no vende productos, no procesa compras ni maneja pagos.

14. Modelo de datos y entidades

14.1. Entidades del foro

Tabla	Propósito	Atributos principales
users	Cuentas de la comunidad	id, first_name, last_name, username, email, password, bio, marcas de tiempo
categories	Catálogo de categorías	id, name, slug, description, marcas de tiempo
publications	Platos y recetas	id, user_id, category_id, type, title, description, image_url, ingredients, instructions, marcas de tiempo
likes	Me gusta de los miembros	id, user_id, publication_id, marcas de tiempo
ratings	Valoraciones de los	id, user_id, publication_id, score, marcas de

Tabla	Propósito	Atributos principales
	miembros	tiempo
comments	Comentarios de los miembros	id, user_id, publication_id, content, marcas de tiempo

14.2. Relaciones y restricciones

- Un usuario tiene muchas publicaciones, me gusta, valoraciones y comentarios.
- Una categoría tiene muchas publicaciones.
- Una publicación tiene muchos me gusta, valoraciones y comentarios.
- Existe un índice único compuesto (user_id, publication_id) en *likes* y en *ratings*.
- La valoración se restringe y valida entre 1 y 5, y el tipo de publicación a *dish* o *recipe*.
- La eliminación de un usuario o de una publicación se propaga a sus registros dependientes según lo configurado.

14.3. Entidades de extensiones

La documentación describe 21 tablas de dominio, además de la tabla técnica *SequelizeMeta* de Sequelize.

Área	Tablas	Propósito
Foro	users, categories, publications, likes, ratings, comments	Funcionalidad principal
Control de acceso	roles, permissions, role_permissions, user_roles	Roles y permisos granulares
Identidad	document_types, user_documents	Documentos de identidad
Ubicación	countries, departments, cities, user_addresses	Datos geográficos y direcciones
Descubrimiento	tags, publication_tags, favorites	Etiquetas y publicaciones guardadas
Notificaciones y moderación	notifications, reports	Avisos en la aplicación y reportes de contenido

14.4. Convenciones de base de datos

- Tablas y columnas en *snake_case*; atributos de modelos Sequelize en *camelCase*.
- Los modelos declaran *tableName*, *underscored: true* y *timestamps: true*.
- Claves primarias enteras autoincrementales; marcas de tiempo *created_at* y *updated_at*.
- Las columnas de clave foránea terminan en *_id*.

- Los valores de catálogo usan restricciones únicas; las tablas de unión usan índices únicos compuestos.
- La documentación indica que se siembran siete categorías. Su contenido exacto debe verificarse en el repositorio, porque los seeders no aparecen en la instantánea.

14.5. Decisiones de datos por confirmar

- Si ingredientes e instrucciones permanecen como texto simple o pasan a listas estructuradas.
- Si las imágenes siguen en archivos locales o pasan a almacenamiento externo.
- Si un usuario puede valorar, comentar o dar me gusta a su propia publicación.
- Si las categorías las administra solo un administrador o son datos estáticos.
- Si las tablas de identidad y ubicación son necesarias para el entregable académico.

15. Rutas y páginas de la aplicación

La columna Estado indica si la ruta aparece registrada en los routers de la instantánea revisada. Las rutas pendientes provienen de la documentación del proyecto y deben verificarse o implementarse.

Método	Ruta	Comportamiento	Acceso	Estado
GET	/	Feed de publicaciones recientes	Público	Por verificar
GET	/categories/:slug	Feed por categoría	Público	Por verificar
GET	/publications/:id	Detalle de la publicación	Público	Por verificar
GET	/publications/new	Formulario de nueva publicación	Autenticado	Por verificar
POST	/publications	Crear publicación (multipart)	Autenticado	Por verificar
GET	/publications/:id/edit	Formulario de edición	Autor	Por verificar
POST	/publications/:id/edit	Actualizar publicación	Autor	Por verificar
POST	/publications/:id/delete	Eliminar publicación	Autor	Por verificar
POST	/publications/:id/like	Alternar me gusta	Autenticado	Por verificar
POST	/publications/:id/rating	Crear o actualizar valoración	Autenticado	Por verificar
POST	/publications/:id/comments	Crear comentario	Autenticado	Por verificar
POST	/comments/:id/edit	Editar comentario	Autor del	Por verificar

Método	Ruta	Comportamiento	Acceso	Estado
			comentario	
POST	/comments/:id/delete	Eliminar comentario	Autor del comentario	Por verificar
GET	/users/:id	Perfil público	Público	Por verificar
GET	/profile/edit	Formulario de edición de perfil	Autenticado	Registrada (falta la vista)
POST	/profile/edit	Actualizar el perfil propio	Autenticado	Registrada
GET	/login	Formulario de inicio de sesión	Público	Registrada
POST	/login	Autenticar	Público	Registrada
GET	/register	Formulario de registro	Público	Registrada
POST	/register	Crear cuenta	Público	Registrada
POST	/logout	Eliminar la cookie de autenticación	Autenticado	Registrada

Páginas previstas: inicio y feed, feed por categoría, detalle de publicación, inicio de sesión, registro, creación de publicación, edición de publicación, perfil público, edición de perfil y página de error o no encontrado. La interfaz debe distinguir platos de recetas, mostrar categoría y autor, usar un marcador cuando no hay imagen, mostrar ingredientes e instrucciones solo en recetas, mostrar editar y eliminar solo al propietario, adaptar la navegación según la sesión y mantener diseños adaptables. En la instantánea solo existen las vistas *login.ejs* y *register.ejs*.

16. Configuración, pruebas y plan de trabajo

16.1. Variables de entorno

Variable	Propósito
APP_PORT	Puerto de la aplicación (3377 por defecto)
NODE_ENV	Entorno de ejecución
DB_DIALECT, DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD	Conexión a PostgreSQL
DB_POOL_MAX, DB_POOL_MIN,	Parámetros del pool de conexiones de Sequelize

Variable	Propósito
DB_POOL_ACQUIRE, DB_POOL_IDLE	
AUTH_APP_JWT_SECRET_KEY	Secreto de firma del JWT
AUTH_APP_JWT_EXPIRES_IN	Vigencia del JWT
UPLOAD_MAX_SIZE_MB	Tamaño máximo de las imágenes subidas, en MB

El archivo Docker Compose también usa PGADMIN_EMAIL y PGADMIN_PASSWORD, que no aparecen en el archivo de ejemplo de variables de entorno. Hay que documentarlas o ajustar la configuración.

16.2. Puesta en marcha en desarrollo

1. Instalar dependencias con *npm install*.
2. Crear el archivo *.env* con *npm run env-var* y completar los datos de base de datos y JWT.
3. Levantar PostgreSQL y pgAdmin con *docker compose up -d*.
4. Aplicar migraciones con *npm run migrations:run*.
5. Ejecutar los seeders con *npm run seeds:run*, una vez confirmado que los archivos y la base de datos son los correctos.
6. Iniciar la aplicación con *npm run dev*, disponible en http://localhost:3377.

Otros scripts: *npm start*, *npm run lint*, *npm test* (pruebas en *tests/*.test.js*), *npm run test:setup*, y los comandos para generar, aplicar y revertir migraciones y seeders. Su disponibilidad debe confirmarse en el repositorio completo.

16.3. Pruebas requeridas

El equipo debe establecer o restaurar pruebas automatizadas para:

- **Autenticación:** registro válido e inválido, usuario o correo duplicado, hash de contraseña, inicio de sesión con usuario y con correo, credenciales inválidas, limpieza de la cookie al cerrar sesión y manejo de JWT vencido o inválido.
- **Perfiles:** ver un perfil existente, 404 para uno desconocido, actualizar el propio, rechazar correo duplicado, rechazar actualizaciones sin sesión y que una actualización no afecte a otro usuario.
- **Publicaciones:** crear plato y receta, rechazar receta sin ingredientes o instrucciones, rechazar categoría inválida, ver, editar y eliminar las propias, rechazar editar o eliminar las ajenas, manejar la ausencia de imagen y rechazar formatos o tamaños no permitidos.
- **Categorías:** feed solo con publicaciones de la categoría y 404 para un identificador desconocido.
- **Me gusta y valoraciones:** agregar, quitar, evitar duplicados, verificar el conteo, crear y actualizar valoración, rechazar puntajes fuera de 1 a 5 y verificar el promedio.

- **Comentarios:** crear, rechazar vacíos o demasiado largos, y editar o eliminar solo los propios.
- **Base de datos:** las migraciones crean todas las tablas, las claves foráneas y los borrados en cascada son correctos, se aplican las restricciones únicas, las asociaciones cargan bien y los seeders pueden repetirse.

16.4. Escenario de aceptación manual

Como mínimo, el equipo debe poder demostrar este recorrido completo:

1. Registrar un usuario nuevo e iniciar sesión.
2. Crear una publicación de tipo plato y otra de tipo receta.
3. Recorrer el feed y una categoría.
4. Abrir una receta y revisar sus ingredientes e instrucciones.
5. Dar me gusta, valorar y comentar una publicación.
6. Editar y eliminar un comentario propio.
7. Editar y eliminar una publicación propia.
8. Abrir un perfil público y editar el perfil propio.
9. Cerrar sesión y confirmar que las acciones restringidas exigen autenticación.

16.5. Plan de trabajo priorizado

Prioridad	Actividades
0. Verificación y estabilización de la base	Comparar la rama real con la instantánea, reconciliar la documentación con rutas, controladores, vistas, pruebas y seeders, verificar la inicialización desde una base vacía, comprobar que *npm test* y *npm run lint* funcionan, confirmar las variables de entorno y la ruta raíz con el destino del cierre de sesión.
1. Flujos centrales del foro	Feed, feed por categoría, detalle, crear, editar y eliminar publicaciones, carga de imágenes, me gusta, valoración con promedio, comentarios, perfil público, vista de edición de perfil, verificaciones de propiedad y pruebas de integración. Se recomienda incluir aquí la protección CSRF, porque los formularios con cookie ya la necesitan.
2. Administración y comunidad	Middleware de autorización por roles y permisos, panel de administración, reportes y su revisión, etiquetas y favoritos.
3. Perfil y participación	Documentos de usuario, direcciones con selectores, notificaciones, búsqueda y paginación del feed, recuperación de contraseña y verificación de correo.
4. Mejoras operativas	Panel de métricas y auditoría, almacenamiento externo de imágenes, notificaciones por correo o push, pruebas de extremo a extremo en navegador y mantenimiento de dependencias.

16.6. Entregables sugeridos

- Informe de verificación del repositorio.
- Diagrama de arquitectura actualizado.
- Backlog de requisitos confirmado.
- Flujos centrales de la aplicación completos.
- Migraciones y datos semilla.
- Pruebas automatizadas y evidencia de su ejecución.
- README e instrucciones de instalación actualizados.
- Demostración final del escenario de extremo a extremo.
- Lista de problemas conocidos y trabajo futuro.

16.7. Buenas prácticas de desarrollo

- Trabajar con ramas por funcionalidad y no versionar secretos ni archivos *.env*.
- Añadir pruebas por cada flujo de negocio nuevo y migraciones por cada cambio de esquema.
- Conservar las verificaciones de propiedad y validar todas las entradas en el servidor.
- Actualizar el README y la documentación de la base de datos cuando cambie el comportamiento.
- Mantener excluido el dominio de comercio y no implementar los módulos extendidos solo porque sus tablas existan.

17. Matriz de trazabilidad

La matriz relaciona cada capacidad con sus requisitos, historias de usuario y entidades principales.

Capacidad	Requisitos	Historias	Entidades principales
Registro	RF-01 a RF-04, RF-11	HU-01	users, roles, user_roles
Inicio y cierre de sesión	RF-05 a RF-10	HU-02, HU-03	users
Perfil público	RF-13, RF-14	HU-05	users, publications
Edición de perfil	RF-15 a RF-18	HU-04	users
Control de acceso y propiedad	RF-10, RF-17, RF-28, RF-49	HU-06	users, publications, comments
Feed	RF-19, RF-25	HU-07	publications, users, categories
Publicación de platos	RF-20, RF-21, RF-23, RF-29, RF-30	HU-08	publications, users, categories

Capacidad	Requisitos	Historias	Entidades principales
Publicación de recetas	RF-22, RF-24, RF-29, RF-30	HU-09	publications, users, categories
Detalle y gestión de publicaciones	RF-25 a RF-28	HU-10, HU-11	publications, users
Categorías	RF-31 a RF-34	HU-12	categories, publications
Búsqueda y paginación	RF-36	HU-13	publications
Me gusta	RF-37 a RF-40	HU-14	users, publications, likes
Valoraciones	RF-41 a RF-44	HU-15	users, publications, ratings
Comentarios	RF-45 a RF-50	HU-16, HU-17	users, publications, comments
Roles, permisos y administración	RF-35, RF-51 a RF-54	HU-22	roles, permissions, user_roles, role_permissions, tags
Favoritos	RF-56	HU-18	favorites, users, publications
Notificaciones	RF-57	HU-19	notifications, users
Reportes y moderación	RF-55, RF-58	HU-20, HU-21	reports, users, publications, comments
Identidad y ubicación	RF-59, RF-60	HU-23	document_types, user_documents, countries, departments, cities, user_addresses